

Arbres

Partie II

1. Rappels

1.1 Préambule

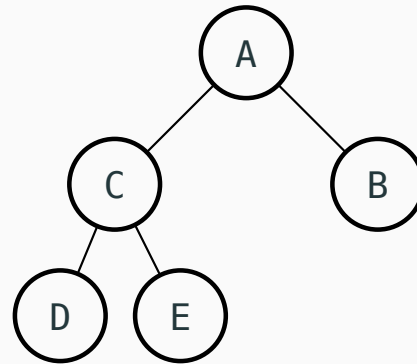
Nous rappelons qu'un **arbre** est un graphe connexe acyclique.

De plus, un arbre peut être **enraciné**, c'est à dire qu'on attribue à un noeud le rôle de racine.

Note

Ainsi, nous retrouverons dans ce cas, la notion de **parent** et **enfants** d'un noeud. On rappelle que dans cette situation, un noeud a un *unique* parent et *plusieurs* enfants.

Voici un exemple d'arbre enraciné en A :



Liste d'adjacence : $\{A:[C, B], B:[], C:[D, E], D:[], E:[]\}$

Nous travaillerons par la suite **exclusivement** avec des **arbres enracinés**.

2. Arbres binaires

2.1 Arbres binaires

Definition 2.1.1 (Arbre binaire)

Un **arbre binaire** est un arbre dont chaque noeud possède au plus 2 enfants.

Proposition 2.1.2 (Arbre binaire parfait)

On dit qu'un arbre est **parfait** si chaque noeud, en dehors des feuilles, a 2 enfants.

Proposition 2.1.3 (Arbre binaire complet)

On dit qu'un arbre est **complet** si tous les niveaux sont totalement remplis, sauf éventuellement le dernier, qui est obligatoirement rempli de gauche à droite sans laisser de trou.

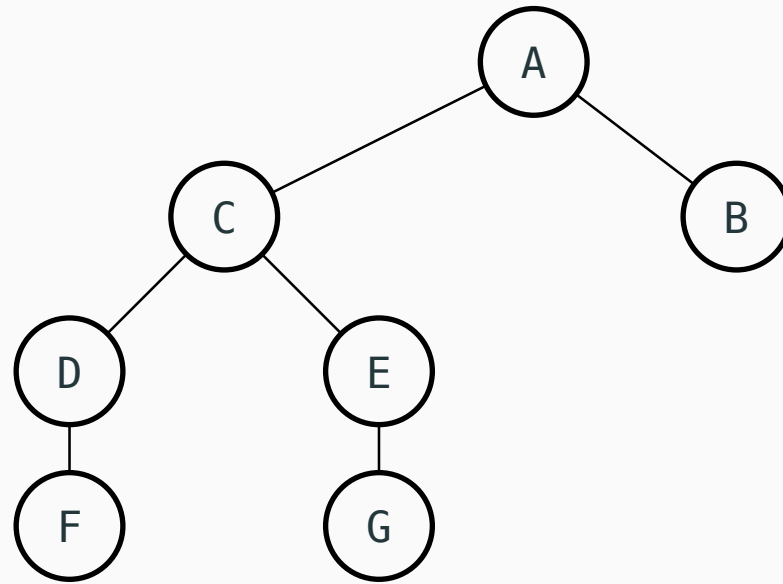
2.1 Arbres binaires

Ce type d'arbre est très récurrent, car ils permettent de simuler pleins de problèmes décisionnels et de découpage des données.

Attention

On supposera par la suite qu'un tel arbre possède un noeud gauche et droit, et éventuellement une valeur.

2.2 Exemple d'arbre binaire



Liste d'adjacence : $\{A:[C, B], B:[], C:[D, E], D:[F], E:[G], F:[], G:[]\}$

2.3 Parcours

Un problème classique avec les arbres est de parcourir tout les noeuds de l'arbre.

Il existe différent type de parcours classiques :

- Parcours préfixe / infixé / suffixe
- Parcours en largeur

Nous verrons les parcours préfixes et suffixes dans le cadre d'un arbre binaire, mais il est aisément possible de le généraliser à tout les arbres.

2.4 Parcours préfixe / infixe / suffixe

Ce parcours est le plus simple. Il est implémenté de manière récursif, et on peut facilement changer entre préfixe, infixe et suffixe :

```
def parcours(node): # Parcours préfixe
    if node == None:
        return
    parcours(node.left)
    print(node.value)
    parcours(node.right)
```

L'idée est que, à chaque noeud, on peut décider de traiter :

- Soit le sous-arbre de gauche
- Soit le sous-arbre de droite
- Soit le noeud qui fait la jointure de ces 2 arbres

2.4 Parcours préfixe / infixe / suffixe

Ainsi, selon les différents modes de parcours, on a les choix suivants :

- Préfixe : sous-arbre de gauche, noeud, sous-arbre de droite
- Infixe : noeud, sous-arbre de gauche, sous-arbre de droite
- Suffixe : sous-arbre de droite, noeud, sous-arbre de gauche

2.5 Parcours en largeur

Ce parcours consiste à passer en revue chaque noeud, couche par couche. Pour obtenir ce parcours, il suffit de faire un *BFS* depuis la racine :

```
from collections import deque # Double ended queue
def parcours_largeur(s):

    queue = deque([s])

    while queue:
        node = queue.pop()
        # Processus
        if node.left != None:
            queue.appendleft(node.left)
        if node.right != None:
            queue.appendleft(node.right)
```

2.5 Parcours en largeur

Voici un schéma du parcours sur cet arbre:

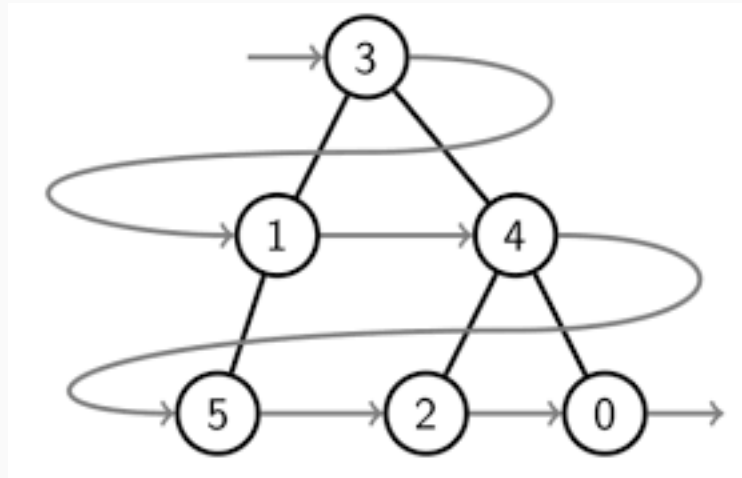


Figure 1: Parcours en largeur

3. Arbres binaires de recherche (ABR)

3.1 Introduction

Definition 3.1.1 (Arbre binaire de recherche)

Un **arbre binaire de recherche** (*ABR* en français et *BST* en anglais), est un arbre où chaque noeud possède une valeur.

Ces valeurs respectent les conditions suivantes, pour chaque noeud avec une valeur A :

- Toutes les valeurs dans le sous-arbre droit sont strictement supérieur à A .
- Toutes les valeurs dans le sous-arbre gauche sont strictement inférieur à A .

Note

On peut régler le problème où on a plusieurs occurrences de la même valeur, en ajoutant une seconde valeur à chaque noeud, indiquant son nombre d'occurrences.

3.2 Efficacité ?

Proposition 3.2.1 (Complexité optimale)

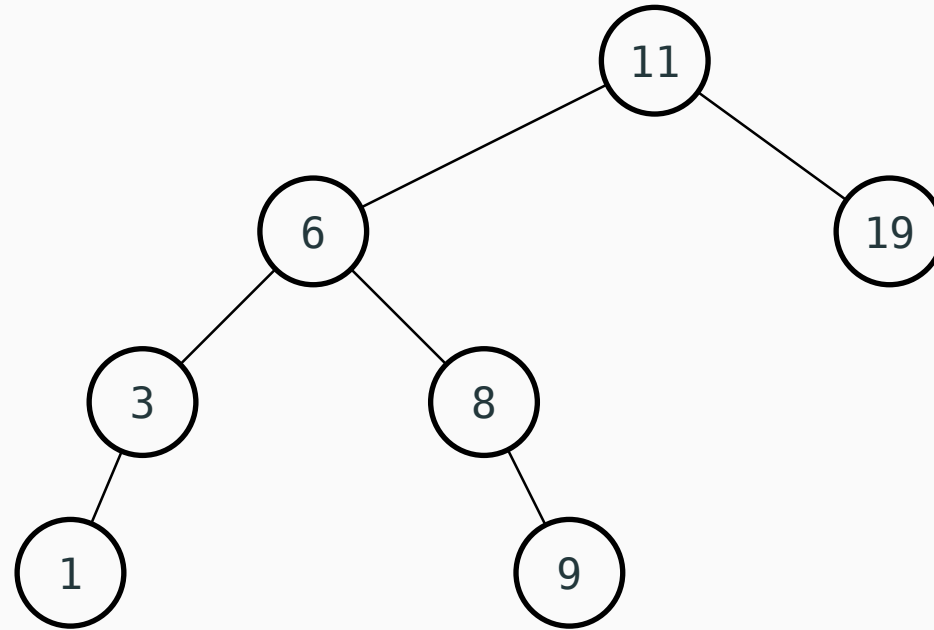
En temps normal, si l'arbre est **complet**, les opérations seront en $O(\log n)$. C'est ce qu'il fait toute la puissance de cette structure.

Nous ne chercherons pas implémenter cette structure, pour plusieurs raisons :

- Elle est implémentée nativement en C++ et Java
- Les opérations implémentées en Python sont trop lentes pour un usage ordinaire

Il s'agit ici de voir un peu leur fonctionnement.

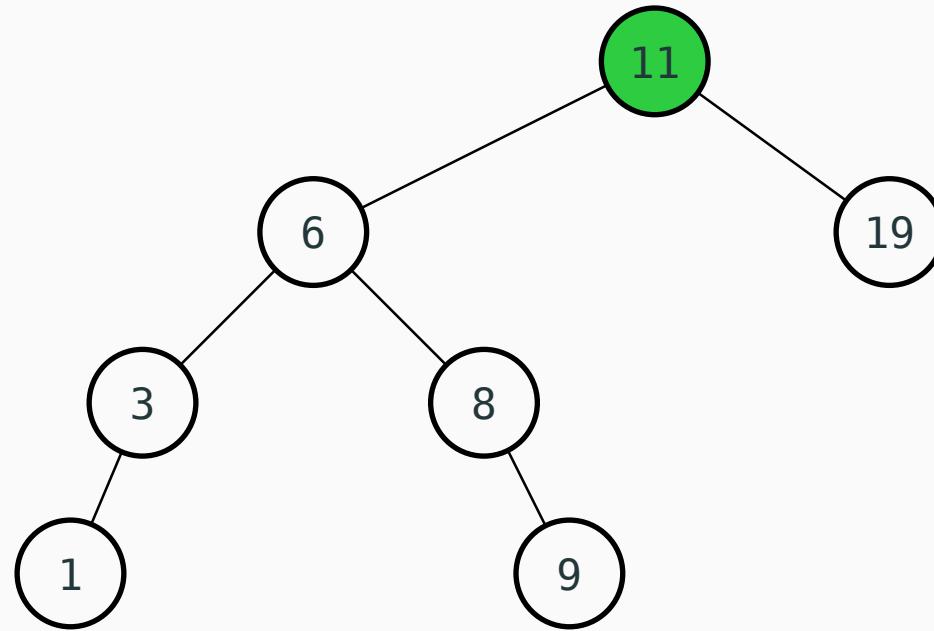
3.3 Trouver un élément



Supposons qu'on cherche le nombre 8.

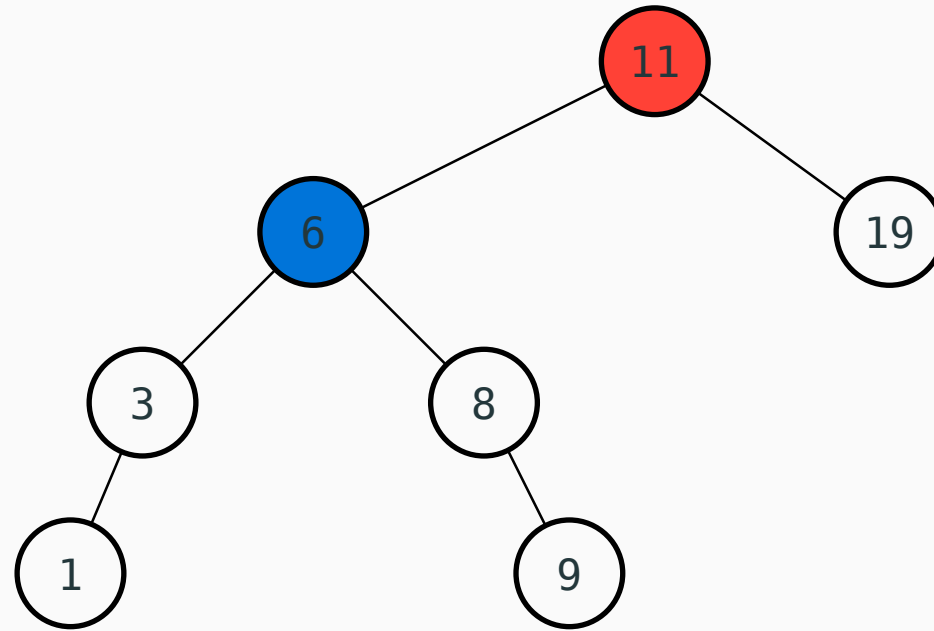
On va exploiter l'ordre des éléments.

3.3 Trouver un élément



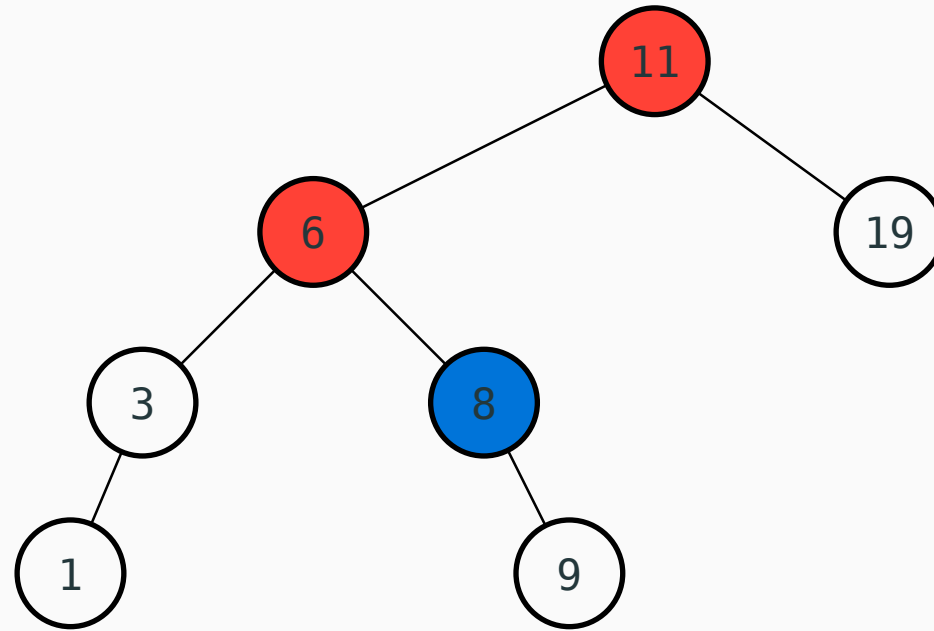
On commence à la racine, qui a une valeur 11. Comme $11 > 10$, on se déplace dans le noeud de gauche.

3.3 Trouver un élément



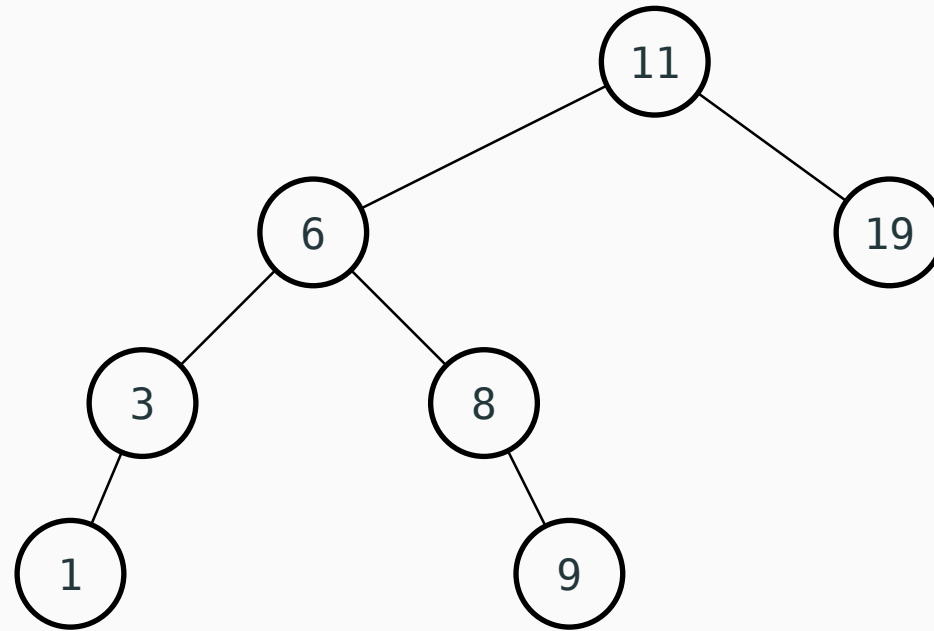
Le noeud a une valeur 6. Comme $6 < 10$, on se déplace dans le noeud de droite.

3.3 Trouver un élément



Le noeud a une valeur 8, on a donc fini le processus.

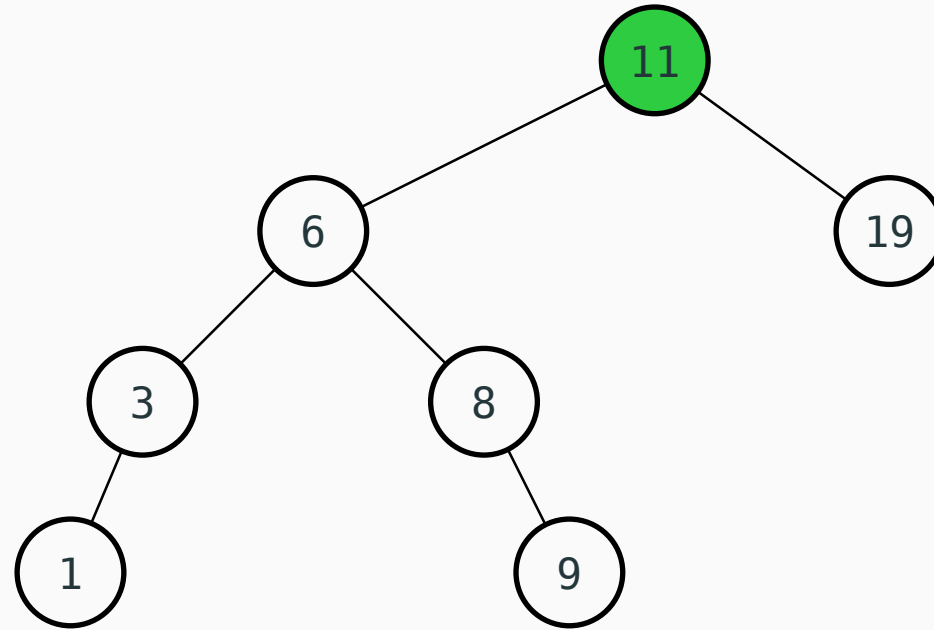
3.4 Ajout d'éléments



Supposons qu'on cherche à ajouter 10 à cet ABR.

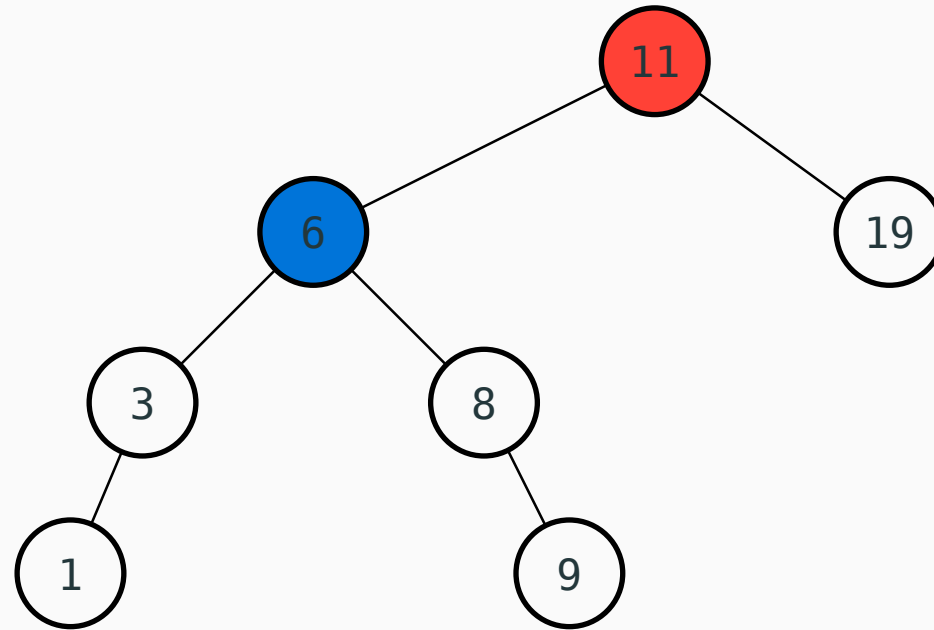
On va exploiter l'ordre des éléments.

3.4 Ajout d'éléments



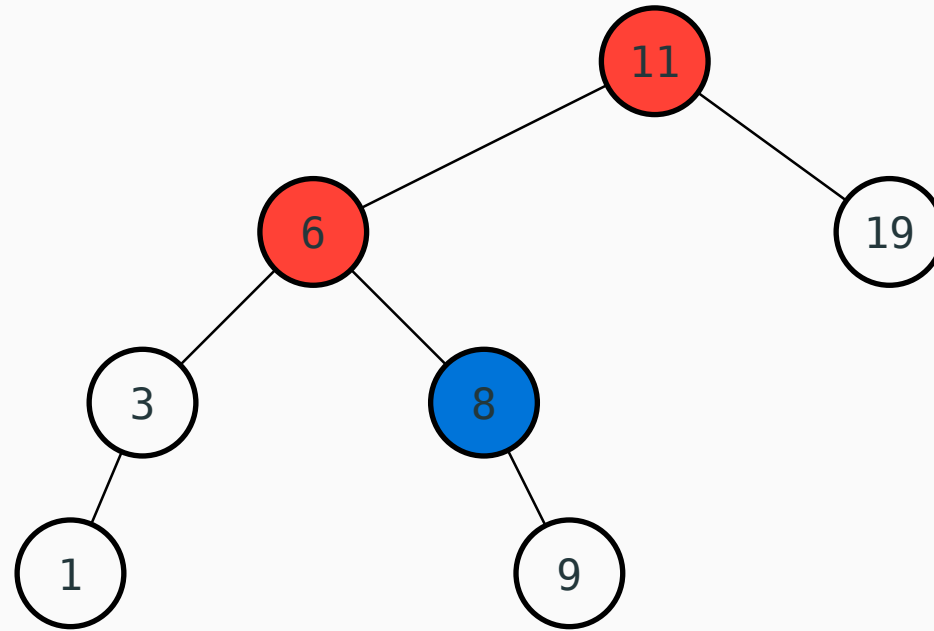
On commence à la racine, qui a une valeur 11. Comme $11 > 10$, on se déplace dans le noeud de gauche.

3.4 Ajout d'éléments



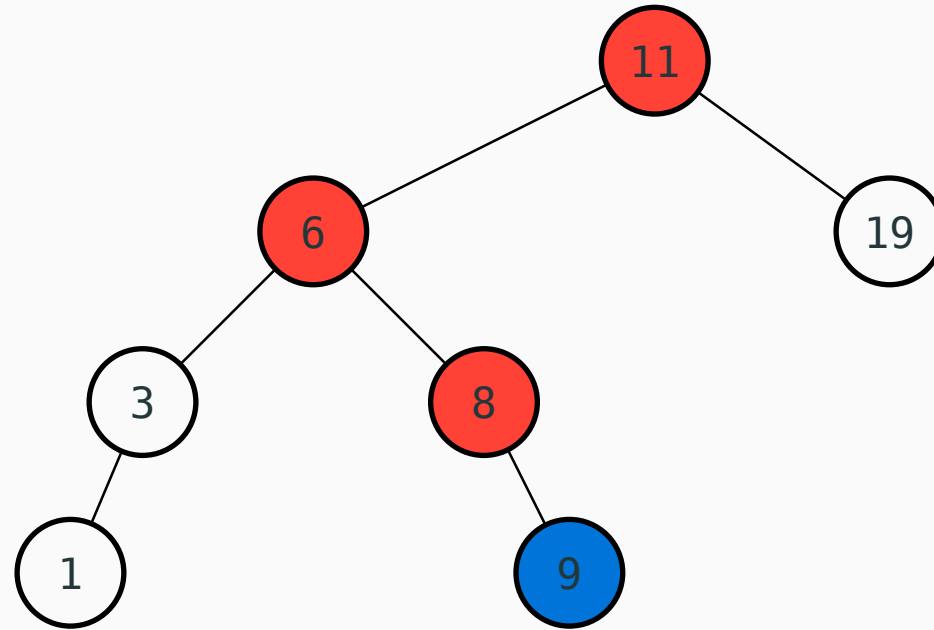
Le noeud a une valeur 6. Comme $6 < 10$, on se déplace dans le noeud de droite.

3.4 Ajout d'éléments



Le noeud a une valeur 8. Comme $8 < 10$, on se déplace dans le noeud de droite.

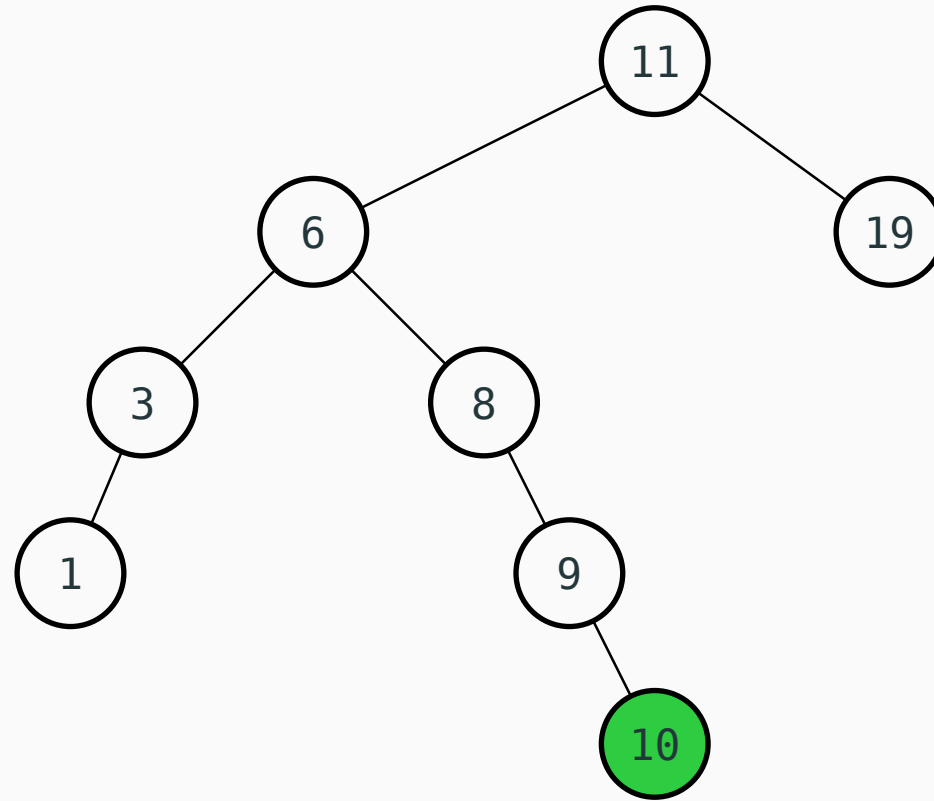
3.4 Ajout d'éléments



Le noeud a une valeur 9. Comme $9 < 10$, on se déplace dans le noeud de droite.

Sauf qu'il n'y a plus de noeud droit, donc on peut l'insérer.

3.4 Ajout d'éléments



3.5 Autres opérations

Il y a possibilité de faire d'autres opérations comme la suppression de noeud ou la fusion, mais nous ne l'aborderons pas dans ce cours.

Voici le résumé des opérations possibles sur cette structure de données :

Opération	Complexité
Accès et modification du k -ème élément	$\Theta(\log n)$
Insertion d'élément	$\Theta(\log n)$
Suppression d'éléments	$\Theta(\log n)$
Recherche d'éléments	$\Theta(\log n)$

L'accès indexé est aisément faisable, en comptant à chaque noeud, le nombre de noeud du sous-arbre gauche et droit.

3.6 Comment passer de $\Theta(\log n)$ à $\mathcal{O}(\log n)$?

En temps normal, si on utilise un simple *ABR*, on est confronté à un problème de complexité si l'arbre obtenu est dégénéré. Car les opérations seront de l'ordre de $\mathcal{O}(n)$.

Definition 3.6.1 (Arbre équilibré)

Un arbre binaire est **équilibré** si à chaque noeud, la différence des hauteurs des 2 sous-arbres est entre -1 et 1 .

Il y a donc possibilité de rendre un *ABR* équilibré, grâce à diverses algorithmes et en maintenant cette condition invariante.

Par exemple, les *ABR* équilibrés les plus communs sont :

- Les arbres *AVL* (**A**delson-**V**elsky et **L**andis)
- Les arbres rouge-noir (bicolore)

3.7 Que faire en pratique ? (En Python)

En Python, nous n'avons pas cette structure par défaut, il faut en importer une que vous pourrez retrouver [ici](#).

Veuillez noter que ce n'est pas un *ABR*, cette structure représente les données en segments, ce qui permet de proposer une complexité $\mathcal{O}\left(n^{\frac{1}{4}}\right)$.

Cette complexité est optimale durant une compétition, et vous êtes autorisés à la copier coller.

3.8 Que faire en pratique ? (En C++)

Les utilisateurs de C++ sont bien servis, car la bibliothèque standard donne accès à des *ABR* équilibrés (même des arbres en général) ... à condition d'être sur *GNU C++*.

GNU C++ donne accès à un module qui s'appelle *PBDS* (Policy Based Data Structures) qui propose un *ABR* équilibré :

```
#include <bits/stdc++.h> // Sous réserve d'avoir l'installation MSYS2
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace __gnu_pbds;

#define ordered_set tree<ll, null_type, std::less<ll>, rb_tree_tag, tree_order_statistics_node_update>
```

La particularité, c'est qu'on peut stocker une seule occurrence d'un élément.

4. Crédits

Auteurs et images : Bilel MAHDJOUR

Pour le **Bureau des technologies** de **Polytech Nice Sophia**

5. Sources et ressources

- [CSES Book](#)
- [Usaco - Tree algorithms](#)